HW6_Problem1

January 28, 2026

1 Problem 1

1.0.1 In this assignment, you will work with a filtered version of the Fashion MNIST dataset to explore the ability of autoencoders to compress and reconstruct image data. You will build and evaluate two autoencoder variants and compare their performance based on various experimental setups. Make sure to print any output you consider important to support your observations.

1.1 1.1

1.1.1 Load the Fashion MNIST dataset from the tensorflow.keras.datasets module. Keep only the instances that belong to the following three classes: 'Sandal', 'Sneaker', and 'Ankle boot'. Split the filtered dataset into a training set (5/7), a validation set (1/7), and a test set (1/7). Make sure the split uses stratified sampling to preserve class balance. Scale all subsets so pixel values are in the range [0, 1]. Print the class distribution of each subset with clear, labeled messages.

Load Fashion MNIST

[1]:
```
from tensorflow.keras.datasets import fashion_mnist

(X_train_full, y_train_full), (X_test_full, y_test_full) = fashion_mnist.
 ↪load_data()
```

Concatenate to extend the dataset and create custom splits.

[2]:
```
import numpy as np
X_full = np.concatenate([X_train_full, X_test_full], axis=0)
y_full = np.concatenate([y_train_full, y_test_full], axis=0)
```

Keep only classes: Sandal (5), Sneaker (7), Ankle boot (9) (The mapping of class indices to labels is standardized and documented)

[3]:
```
selected_classes = [5, 7, 9]
mask = np.isin(y_full, selected_classes)
X_full = X_full[mask]
y_full = y_full[mask]
```

Define the split sizes. Consider the full dataset split in 7 parts.

[4]:
```
train_size = 5/7
valid_size = 1/7
test_size = 1/7
```

Export the test set by splitting on the full sets. We keep $\frac{1}{7} \cdot dataset$.

[5]:
```
from sklearn.model_selection import train_test_split

X_temp, X_test, y_temp, y_test = train_test_split(X_full, y_full,␣
 ↪test_size=test_size, random_state=42, stratify=y_full)
```

Having kept $\frac{1}{7} \cdot dataset$, we now want to export from the remaining $\frac{6}{7} \cdot dataset$, another $\frac{1}{7} \cdot dataset$. Therefore we will use a ratio of $\frac{\frac{1}{7}}{\frac{5}{7}+\frac{1}{7}} = \frac{\frac{1}{7}}{\frac{6}{7}} = \frac{1}{6} \cdot temp\_dataset$.

[6]:
```
valid_ratio_adjusted = valid_size / (train_size + valid_size)
X_train, X_valid, y_train, y_valid = train_test_split(X_temp, y_temp,␣
 ↪test_size=valid_ratio_adjusted, random_state=42, stratify=y_temp)
```

Normalize pixel values 0-255 to 0-1. Setting the datatype explicitly is important for TensorFlow/Keras models since they expect float inputs to perform computations efficiently.

[7]:
```
X_train = X_train.astype("float32") / 255.
X_valid = X_valid.astype("float32") / 255.
X_test = X_test.astype("float32") / 255.
```

Print dataset shapes

[8]:
```
print(f'Train set shape: {X_train.shape}, Labels shape: {y_train.shape}')
print(f'Validation set shape: {X_valid.shape}, Labels shape: {y_valid.shape}')
print(f'Test set shape: {X_test.shape}, Labels shape: {y_test.shape}')
```

```
Train set shape: (15000, 28, 28), Labels shape: (15000,)
Validation set shape: (3000, 28, 28), Labels shape: (3000,)
Test set shape: (3000, 28, 28), Labels shape: (3000,)
```

Verify that the stratified splitting preserved class balance

[9]:
```
import pandas as pd

def print_class_distribution(y, name):
    class_counts = pd.Series(y).value_counts().sort_index()
    class_names = {5: 'Sandal', 7: 'Sneaker', 9: 'Ankle boot'}
    print(f'\nClass distribution in {name} set:')
    for label, count in class_counts.items():
        print(f'{class_names[label]:<12}: {count}')

print_class_distribution(y_train, "Train")
print_class_distribution(y_valid, "Validation")
print_class_distribution(y_test, "Test")
```

```
Class distribution in Train set:
Sandal      : 5000
Sneaker     : 5000
Ankle boot  : 5000

Class distribution in Validation set:
Sandal      : 1000
Sneaker     : 1000
Ankle boot  : 1000

Class distribution in Test set:
Sandal      : 1000
Sneaker     : 1000
Ankle boot  : 1000
```

1.2 1.2

1.2.1 Build a stacked autoencoder with the following architecture: an encoder and a decoder part of two fully-connected (Dense) layers each. Use ReLU activation for all hidden layers, and sigmoid activation for the output layer. Flatten the input before feeding it to the encoder, and reshape the output to match the original image shape (28×28). Set the latent space (codings) size to 50, and all other hidden layers to 256 units. Compile the model using: Nadam optimizer with learning rate 10^{-4}, adding binary cross-entropy as the loss function. Print the model summary of this autoencoder.

Define the constants

[10]:
```
img_height, img_width = 28, 28
flattened_dim = img_height * img_width
hidden_units = 256
latent_dim = 50
```

Build the stacked autoencoder

[11]:
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Flatten, Dense, Reshape

autoencoder = Sequential([
    Input(shape=(28, 28)),

    # Encoder
    Flatten(),
    Dense(hidden_units, activation='relu'),
    Dense(latent_dim, activation='relu'),

    # Decoder
    Dense(hidden_units, activation='relu'),
```

```
    Dense(flattened_dim, activation='sigmoid'),
    Reshape((28, 28))

])
```

Compile

[12]:
```
from tensorflow.keras.optimizers import Nadam

autoencoder.compile(optimizer=Nadam(learning_rate=10**(-4)),␣
 ↪loss='binary_crossentropy')

autoencoder.summary()
```

```
Model: "sequential"
```

```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━┓
┃ Layer (type)                    ┃ Output Shape                          ┃       Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━┩
│ flatten (Flatten)               │ (None, 784)                           │             0 │
├─────────────────────────────────┼───────────────────────────────────────┼───────────────┤
│ dense (Dense)                   │ (None, 256)                           │       200,960 │
├─────────────────────────────────┼───────────────────────────────────────┼───────────────┤
│ dense_1 (Dense)                 │ (None, 50)                            │        12,850 │
├─────────────────────────────────┼───────────────────────────────────────┼───────────────┤
│ dense_2 (Dense)                 │ (None, 256)                           │        13,056 │
├─────────────────────────────────┼───────────────────────────────────────┼───────────────┤
│ dense_3 (Dense)                 │ (None, 784)                           │       201,488 │
├─────────────────────────────────┼───────────────────────────────────────┼───────────────┤
│ reshape (Reshape)               │ (None, 28, 28)                        │             0 │
└─────────────────────────────────┴───────────────────────────────────────┴───────────────┘
```

```
 Total params: 428,354 (1.63 MB)

 Trainable params: 428,354 (1.63 MB)

 Non-trainable params: 0 (0.00 B)
```

1.3 1.3

1.3.1 Now, build a second autoencoder that is identical in architecture to the one in Question 2, add l1 regularization to the latent space (the bottleneck layer), and use an activity regularizer with weight 10^{-6}. Print the model summary for this sparse autoencoder.

Define constants

[13]:
```
img_height, img_width = 28, 28
flattened_dim = img_height * img_width
hidden_units = 256
latent_dim = 50
activity_reg_weight = 10**(-6)
```

Build the sparse autoencoder

[14]:
```
from tensorflow.keras.regularizers import l1

sparse_autoencoder = Sequential([
    Input(shape=(28, 28)),

    # Encoder
    Flatten(),
    Dense(hidden_units, activation='relu'),
    Dense(latent_dim, activation='relu',␣
 ↪activity_regularizer=l1(activity_reg_weight)),

    # Decoder
    Dense(hidden_units, activation='relu'),
    Dense(flattened_dim, activation='sigmoid'),
    Reshape((28, 28))
])
```

Compile

[15]:
```
sparse_autoencoder.compile(optimizer=Nadam(learning_rate=10**(-4)),␣
 ↪loss='binary_crossentropy')

sparse_autoencoder.summary()
```

```
Model: "sequential_1"
```

```
 Layer (type)                Output Shape              ␣
↪Param #

 flatten_1 (Flatten)         (None, 784)               ␣
↪  0
```

```
dense_4 (Dense)                      (None, 256)                 200,960

dense_5 (Dense)                      (None, 50)                  12,850

dense_6 (Dense)                      (None, 256)                 13,056

dense_7 (Dense)                      (None, 784)                 201,488

reshape_1 (Reshape)                  (None, 28, 28)              0
```

Total params: 428,354 (1.63 MB)

Trainable params: 428,354 (1.63 MB)

Non-trainable params: 0 (0.00 B)

1.4 1.4

1.4.1 Train both autoencoders (from Q2 and Q3) for up to 50 epochs. Use early stopping with patience=5, and min_delta equal to 10^{-2}, monitoring the validation loss. Train each model with two different batch sizes: 32 and 256. This gives you four experiments in total (2 models × 2 batch sizes). After running your experiments, plot the training and validation loss curves for all of them. Add your comments on the differences observed between the models and batch sizes.

Set up early stopping

[16]:
```
from tensorflow.keras.callbacks import EarlyStopping

early_stopping_cb = EarlyStopping(monitor='val_loss', patience=5,␣
↪min_delta=1e-2, restore_best_weights=True, verbose=1)
```

Flatten inputs for dense autoencoders

[17]:
```
X_train_flat = X_train.reshape(-1, 28, 28)
X_valid_flat = X_valid.reshape(-1, 28, 28)
```

Dictionaries to store models and histories

[18]:
```
from tensorflow.keras.models import clone_model

models = {
    "regular_bs32": clone_model(autoencoder),
    "regular_bs256": clone_model(autoencoder),
    "sparse_bs32": clone_model(sparse_autoencoder),
    "sparse_bs256": clone_model(sparse_autoencoder),
}

histories = {}
```

Compile and train each model

[19]:
```
for key, model in models.items():
    model.compile(optimizer=Nadam(learning_rate=1e-4),␣
 ↪loss='binary_crossentropy')
    batch_size = 32 if "32" in key else 256
    print(f"\nTraining {key} with batch size {batch_size}...\n")

    history = model.fit(
        X_train_flat, X_train_flat,
        epochs=50,
        batch_size=batch_size,
        validation_data=(X_valid_flat, X_valid_flat),
        callbacks=[early_stopping_cb],
        verbose=2
    )

    histories[key] = history
```

```
Training regular_bs32 with batch size 32…

Epoch 1/50
469/469 - 2s - 4ms/step - loss: 0.3526 - val_loss: 0.2796
Epoch 2/50
469/469 - 1s - 2ms/step - loss: 0.2652 - val_loss: 0.2568
Epoch 3/50
469/469 - 1s - 2ms/step - loss: 0.2490 - val_loss: 0.2450
Epoch 4/50
469/469 - 1s - 2ms/step - loss: 0.2401 - val_loss: 0.2381
Epoch 5/50
469/469 - 1s - 2ms/step - loss: 0.2345 - val_loss: 0.2336
Epoch 6/50
469/469 - 1s - 2ms/step - loss: 0.2307 - val_loss: 0.2304
Epoch 7/50
469/469 - 1s - 2ms/step - loss: 0.2277 - val_loss: 0.2278
Epoch 8/50
```

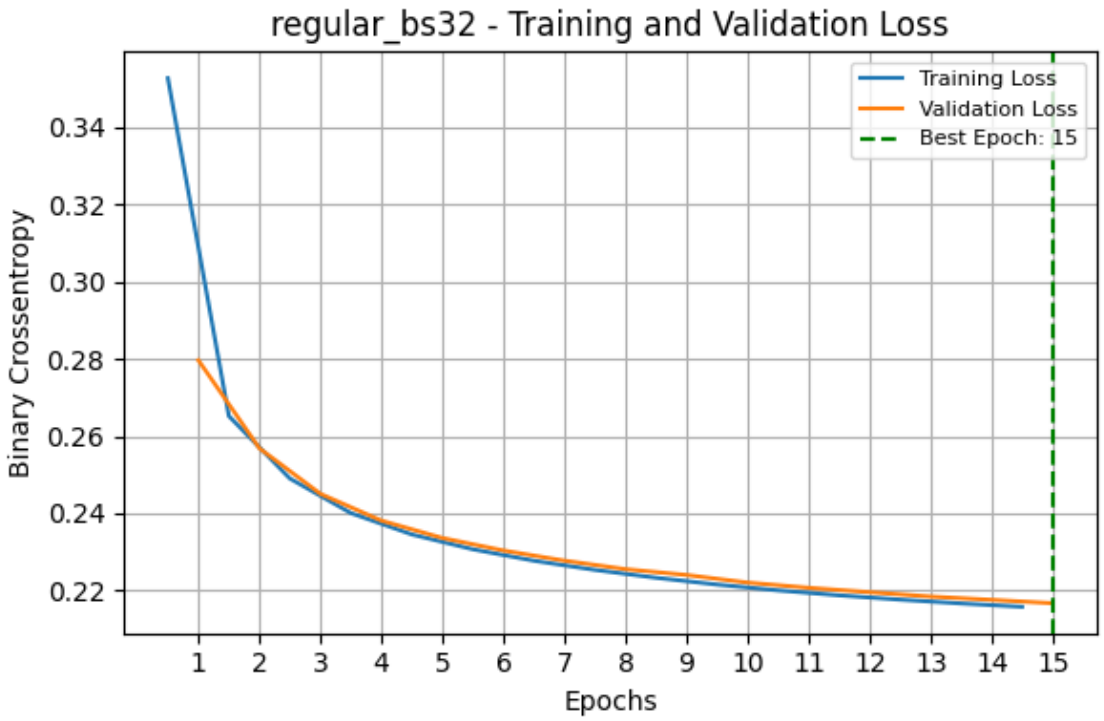
```
469/469 - 1s - 2ms/step - loss: 0.2254 - val_loss: 0.2256
Epoch 9/50
469/469 - 1s - 2ms/step - loss: 0.2233 - val_loss: 0.2241
Epoch 10/50
469/469 - 1s - 2ms/step - loss: 0.2216 - val_loss: 0.2221
Epoch 11/50
469/469 - 1s - 2ms/step - loss: 0.2201 - val_loss: 0.2207
Epoch 12/50
469/469 - 1s - 2ms/step - loss: 0.2188 - val_loss: 0.2196
Epoch 13/50
469/469 - 1s - 2ms/step - loss: 0.2177 - val_loss: 0.2185
Epoch 14/50
469/469 - 1s - 2ms/step - loss: 0.2167 - val_loss: 0.2176
Epoch 15/50
469/469 - 1s - 2ms/step - loss: 0.2158 - val_loss: 0.2167
Epoch 15: early stopping
Restoring model weights from the end of the best epoch: 10.

Training regular_bs256 with batch size 256…

Epoch 1/50
59/59 - 1s - 23ms/step - loss: 0.6200 - val_loss: 0.4789
Epoch 2/50
59/59 - 0s - 5ms/step - loss: 0.3946 - val_loss: 0.3437
Epoch 3/50
59/59 - 0s - 5ms/step - loss: 0.3234 - val_loss: 0.3090
Epoch 4/50
59/59 - 0s - 5ms/step - loss: 0.3002 - val_loss: 0.2935
Epoch 5/50
59/59 - 0s - 5ms/step - loss: 0.2875 - val_loss: 0.2836
Epoch 6/50
59/59 - 0s - 5ms/step - loss: 0.2784 - val_loss: 0.2760
Epoch 7/50
59/59 - 0s - 5ms/step - loss: 0.2717 - val_loss: 0.2704
Epoch 8/50
59/59 - 0s - 5ms/step - loss: 0.2665 - val_loss: 0.2659
Epoch 9/50
59/59 - 0s - 6ms/step - loss: 0.2622 - val_loss: 0.2619
Epoch 10/50
59/59 - 0s - 5ms/step - loss: 0.2585 - val_loss: 0.2584
Epoch 11/50
59/59 - 0s - 5ms/step - loss: 0.2552 - val_loss: 0.2554
Epoch 12/50
59/59 - 0s - 6ms/step - loss: 0.2522 - val_loss: 0.2525
Epoch 13/50
59/59 - 0s - 6ms/step - loss: 0.2496 - val_loss: 0.2501
Epoch 14/50
59/59 - 0s - 6ms/step - loss: 0.2472 - val_loss: 0.2477
```

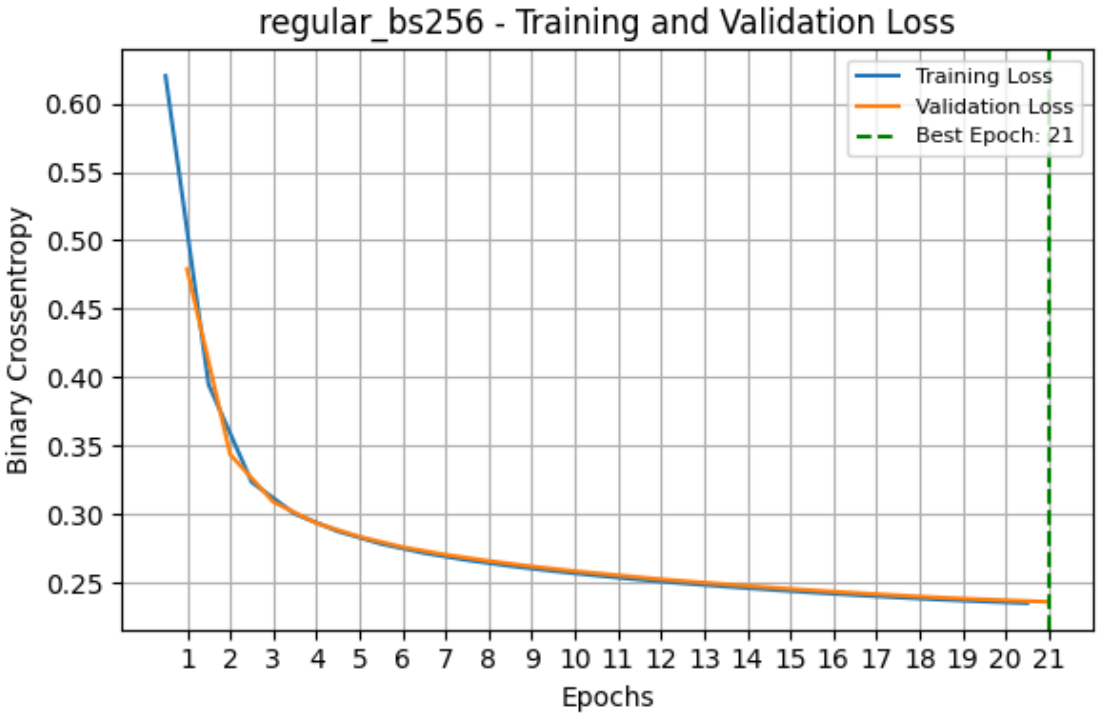
```
Epoch 15/50
59/59 - 0s - 5ms/step - loss: 0.2450 - val_loss: 0.2455
Epoch 16/50
59/59 - 0s - 5ms/step - loss: 0.2429 - val_loss: 0.2435
Epoch 17/50
59/59 - 0s - 5ms/step - loss: 0.2410 - val_loss: 0.2416
Epoch 18/50
59/59 - 0s - 5ms/step - loss: 0.2392 - val_loss: 0.2399
Epoch 19/50
59/59 - 0s - 6ms/step - loss: 0.2376 - val_loss: 0.2384
Epoch 20/50
59/59 - 0s - 6ms/step - loss: 0.2363 - val_loss: 0.2372
Epoch 21/50
59/59 - 0s - 5ms/step - loss: 0.2351 - val_loss: 0.2360
Epoch 21: early stopping
Restoring model weights from the end of the best epoch: 16.

Training sparse_bs32 with batch size 32…

Epoch 1/50
469/469 - 2s - 5ms/step - loss: 0.3578 - val_loss: 0.2842
Epoch 2/50
469/469 - 1s - 2ms/step - loss: 0.2696 - val_loss: 0.2610
Epoch 3/50
469/469 - 1s - 2ms/step - loss: 0.2533 - val_loss: 0.2489
Epoch 4/50
469/469 - 1s - 2ms/step - loss: 0.2438 - val_loss: 0.2416
Epoch 5/50
469/469 - 1s - 2ms/step - loss: 0.2381 - val_loss: 0.2370
Epoch 6/50
469/469 - 1s - 2ms/step - loss: 0.2339 - val_loss: 0.2334
Epoch 7/50
469/469 - 1s - 2ms/step - loss: 0.2306 - val_loss: 0.2304
Epoch 8/50
469/469 - 1s - 2ms/step - loss: 0.2281 - val_loss: 0.2282
Epoch 9/50
469/469 - 1s - 2ms/step - loss: 0.2259 - val_loss: 0.2262
Epoch 10/50
469/469 - 1s - 2ms/step - loss: 0.2241 - val_loss: 0.2245
Epoch 11/50
469/469 - 1s - 2ms/step - loss: 0.2225 - val_loss: 0.2231
Epoch 12/50
469/469 - 1s - 2ms/step - loss: 0.2211 - val_loss: 0.2219
Epoch 13/50
469/469 - 1s - 2ms/step - loss: 0.2200 - val_loss: 0.2208
Epoch 14/50
469/469 - 1s - 2ms/step - loss: 0.2189 - val_loss: 0.2199
Epoch 14: early stopping
```

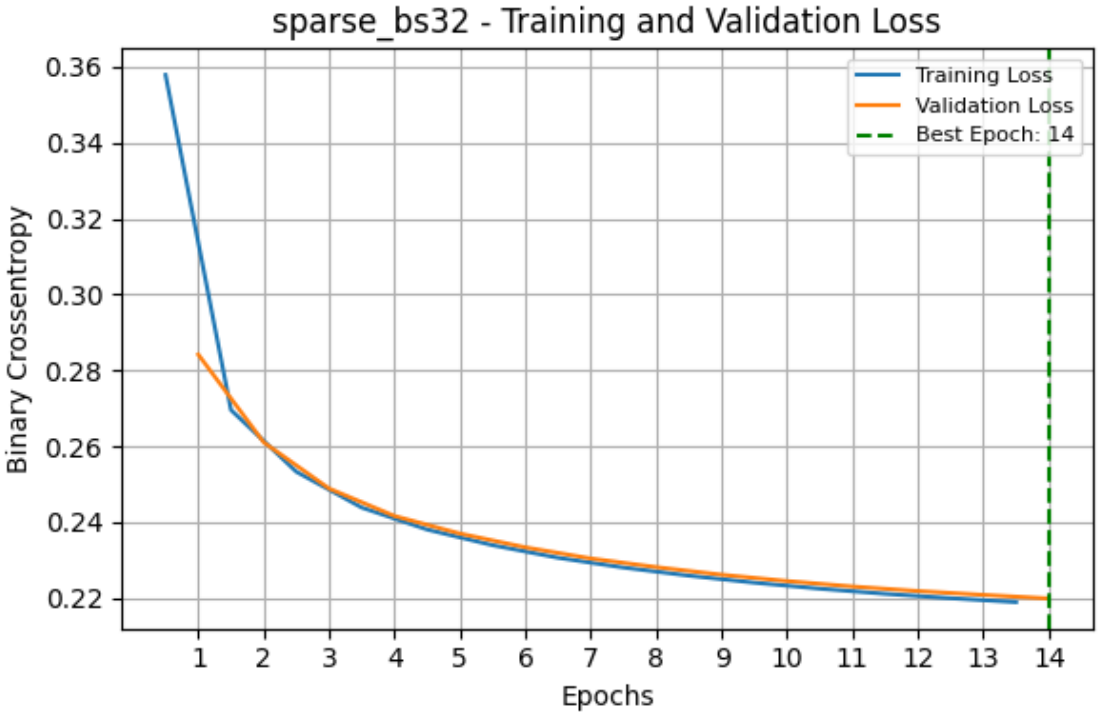
```
Restoring model weights from the end of the best epoch: 9.

Training sparse_bs256 with batch size 256…

Epoch 1/50
59/59 - 1s - 25ms/step - loss: 0.6374 - val_loss: 0.5144
Epoch 2/50
59/59 - 0s - 6ms/step - loss: 0.4262 - val_loss: 0.3730
Epoch 3/50
59/59 - 0s - 7ms/step - loss: 0.3510 - val_loss: 0.3343
Epoch 4/50
59/59 - 0s - 6ms/step - loss: 0.3242 - val_loss: 0.3156
Epoch 5/50
59/59 - 0s - 6ms/step - loss: 0.3086 - val_loss: 0.3029
Epoch 6/50
59/59 - 0s - 6ms/step - loss: 0.2967 - val_loss: 0.2925
Epoch 7/50
59/59 - 0s - 6ms/step - loss: 0.2874 - val_loss: 0.2848
Epoch 8/50
59/59 - 0s - 7ms/step - loss: 0.2806 - val_loss: 0.2790
Epoch 9/50
59/59 - 0s - 7ms/step - loss: 0.2751 - val_loss: 0.2741
Epoch 10/50
59/59 - 0s - 7ms/step - loss: 0.2705 - val_loss: 0.2697
Epoch 11/50
59/59 - 0s - 6ms/step - loss: 0.2662 - val_loss: 0.2656
Epoch 12/50
59/59 - 0s - 7ms/step - loss: 0.2625 - val_loss: 0.2620
Epoch 13/50
59/59 - 0s - 7ms/step - loss: 0.2592 - val_loss: 0.2591
Epoch 14/50
59/59 - 0s - 6ms/step - loss: 0.2563 - val_loss: 0.2563
Epoch 15/50
59/59 - 0s - 7ms/step - loss: 0.2538 - val_loss: 0.2539
Epoch 16/50
59/59 - 0s - 6ms/step - loss: 0.2516 - val_loss: 0.2518
Epoch 17/50
59/59 - 0s - 7ms/step - loss: 0.2496 - val_loss: 0.2498
Epoch 18/50
59/59 - 0s - 7ms/step - loss: 0.2478 - val_loss: 0.2482
Epoch 19/50
59/59 - 0s - 7ms/step - loss: 0.2463 - val_loss: 0.2470
Epoch 20/50
59/59 - 0s - 7ms/step - loss: 0.2449 - val_loss: 0.2455
Epoch 20: early stopping
Restoring model weights from the end of the best epoch: 15.
```

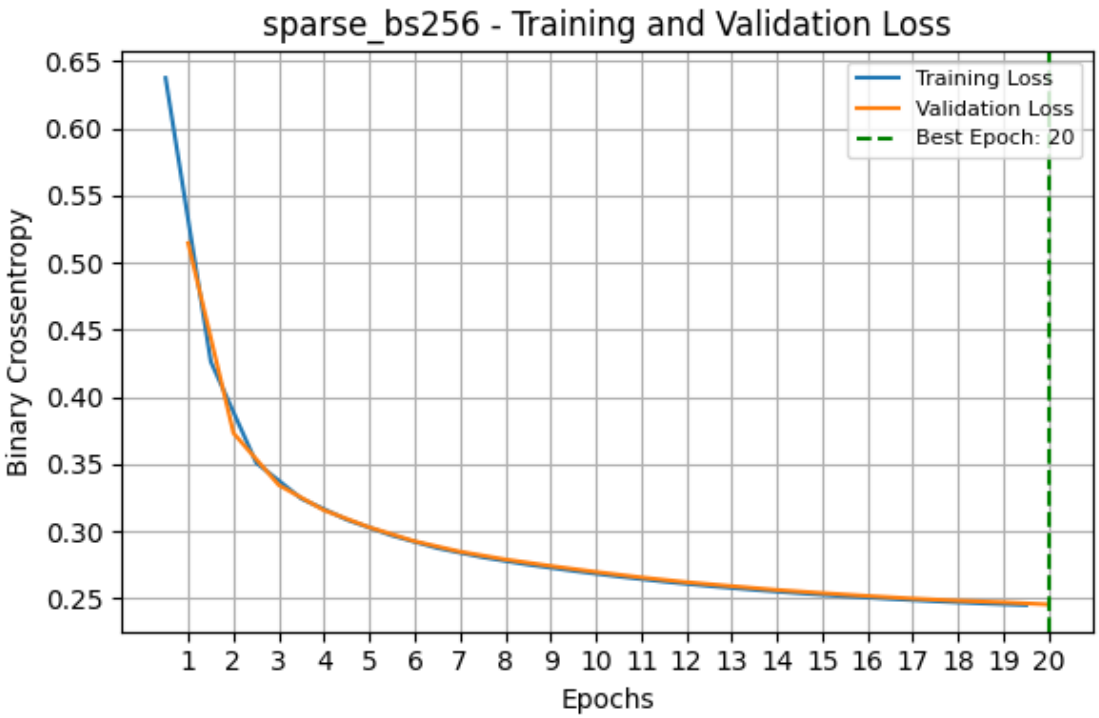
Plot the training and validation losses for all the autoencoders

[21]:
```
import matplotlib.pyplot as plt

for key, history in histories.items():
    epochs = np.arange(len(history.epoch))
    best_epoch = np.argmin(history.history['val_loss'])

    plt.figure(figsize=(6, 4))

    # Plot training and validation loss only
    plt.plot(epochs - 0.5, history.history['loss'], label='Training Loss')
    plt.plot(epochs, history.history['val_loss'], label='Validation Loss')
    plt.axvline(best_epoch, color='green', linestyle='--', label=f'Best Epoch:
 ↪{best_epoch + 1}')
    plt.title(f'{key} - Training and Validation Loss')
    plt.xlabel('Epochs')
    plt.xticks(ticks=epochs, labels=epochs + 1)
    plt.ylabel('Binary Crossentropy')
    plt.grid()
    plt.legend(loc='upper right', fontsize=8)

    plt.tight_layout()
    plt.show()
```




regular_bs256 - Training and Validation Loss
Training Loss
Validation Loss
Best Epoch: 21
Binary Crossentropy
0.60
0.55
0.50
0.45
0.40
0.35
0.30
0.25
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21
Epochs


sparse_bs32 - Training and Validation Loss
Training Loss
Validation Loss
Best Epoch: 14
Binary Crossentropy
0.36
0.34
0.32
0.30
0.28
0.26
0.24
0.22
1 2 3 4 5 6 7 8 9 10 11 12 13 14
Epochs



1.5 1.5

1.5.1 Write a function that compares original images with their reconstructions from an autoencoder. The function should:

1.5.2 • Accept a model, a set of images, and the number of images to display as input.

1.5.3 • Display a grid where the top row shows the original images, and the bottom row shows the reconstructed images.

1.5.4 • Ensure the pixel values are clipped to the range [0, 1] for proper visualization.

1.5.5 Then use this function to display the first 15 test images and their reconstructions from the regular autoencoder trained with a batch size of 256, and the sparse autoencoder trained with a batch size 256.

[22]:
```
def show_reconstructions(model, images, n_images=15):
    """
    Plots original and reconstructed images in a 2-row grid.

    Top row: original images
    Bottom row: reconstructed images
    """
    reconstructions = model.predict(images[:n_images], verbose=0)
```

```
    reconstructions = np.clip(reconstructions, 0, 1)  # Ensure proper range

    plt.figure(figsize=(n_images, 3))
    for i in range(n_images):
        # Original
        plt.subplot(2, n_images, i + 1)
        plt.imshow(images[i], cmap='gray')
        plt.axis('off')

        # Reconstructed
        plt.subplot(2, n_images, i + 1 + n_images)
        plt.imshow(reconstructions[i], cmap='gray')
        plt.axis('off')

    plt.suptitle("Top: Original Images - Bottom: Reconstructions", fontsize=12)
    plt.tight_layout()
    plt.show()
```

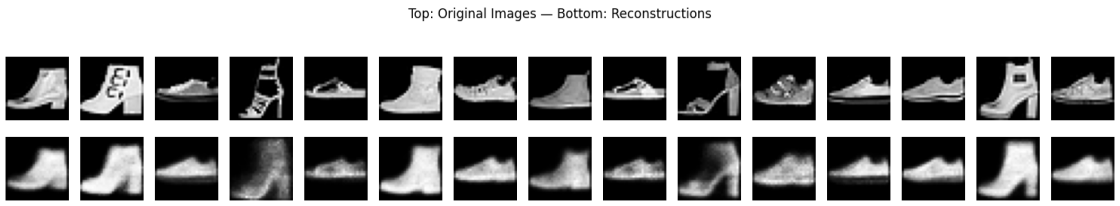
Flatten test images for model input

[23]:
```
X_test_flat = X_test.reshape(-1, 28, 28)
```

Display reconstructions for regular autoencoder (batch size 256)

[24]:
```
print("Regular Autoencoder (batch size 256):")
show_reconstructions(models["regular_bs256"], X_test_flat, n_images=15)
```

```
Regular Autoencoder (batch size 256):
```



Display reconstructions for sparse autoencoder (batch size 256)

[25]:
```
print("Sparse Autoencoder (batch size 256):")
show_reconstructions(models["sparse_bs256"], X_test_flat, n_images=15)
```

```
Sparse Autoencoder (batch size 256):
```

Top: Original Images — Bottom: Reconstructions